

03_SCRIPTS_Y_FUNCIONES

August 21, 2026



1 MÓDULO 03: Escritura de Scripts, Funciones y Reutilización de Módulos

Curso: Python para Ciencia Abierta (CSIC)

Autor: Gustavo Liñán Cembrano

Fechas: 5 a 8 de Octubre de 2026

Tiempo Estimado de Impartición: 1 hora

Módulo: 03 - Python Scripts, User-Defined Functions, Docstrings & Modular Code

1.1 1. Objetivos de Aprendizaje

- **Estructura de Scripts (.py):** Comprender la anatomía de un script ejecutable, separando importaciones, definiciones de funciones y el bloque ejecutable principal (`if __name__ == "__main__":`).
- **Definición de Funciones (def):** Aprender la sintaxis de definición con parámetros posicionales, parámetros por defecto y retornos simples y múltiples (tuplas).
- **Documentación con Docstrings:** Incorporar especificaciones estándar de documentación (`"""..."""`) y consultar la ayuda nativa con `help()`.
- **Funciones Anónimas (lambda):** Utilizar expresiones lambda combinadas con funciones de orden superior (`map()` y `filter()`) para procesamiento funcional eficiente.
- **Reutilización de Módulos:** Crear e importar funciones de archivos propios (`mylib.py`).

2 SESION 3: Escritura de Scripts y Funciones

En esta sesión aprenderás a escribir scripts en Python y a definir tus propias funciones para organizar mejor tu código. Estos conceptos son fundamentales para desarrollar programas reutilizables, legibles y eficientes.

2.0.1 Objetivos:

- Comprender la estructura básica de un script en Python.
- Valorar la importancia de la legibilidad y los comentarios.
- Definir funciones propias con argumentos y valores de retorno.
- Utilizar funciones anónimas (lambdas) para tareas simples.
- Aplicar lo aprendido en actividades prácticas.

@Gustavo Liñán Cembrano

2.1 Escritura de Scripts en Python

Un **script en Python** es un archivo de texto con extensión `.py` que contiene código ejecutable. Se diferencia de un cuaderno Jupyter en que **se ejecuta siempre de arriba a abajo como un todo**, sin celdas separadas.

Piensa en él como si pegáramos todas las celdas de un cuaderno en un único archivo.

2.1.1 Estructura típica de un script

Aunque Python no obliga a un orden fijo, es recomendable seguir una organización clara:

1. **Importaciones:** módulos o librerías que necesitamos.
2. **Definiciones:** funciones (y más adelante clases) que organizan el código.
3. **Bloque principal:** el código que realmente se ejecuta.
Suele ir dentro de:

“python if **name** == “**main**”: # aquí va el código principal

```
[ ]: # script_ejemplo.py

# 1. Importaciones
import math # Librería estándar para cálculos matemáticos

# 2.1 Definiciones (funciones)
def calcular_area_heron(a, b, c):
    """
    Calcula el área de un triángulo dados sus tres lados usando la fórmula de
    ↪ Herón.
    Si los lados no forman un triángulo válido (incluyendo el caso degenerado),
    """
```

```

    devuelve None.
    """
    # Verificación de desigualdad triangular (estricta)
    if not ((a + b > c) and (a + c > b) and (b + c > a)):
        return None # No es un triángulo válido

    # Cálculo con fórmula de Herón
    s = (a + b + c) / 2 # Semiperímetro
    area = math.sqrt(s * (s - a) * (s - b) * (s - c))
    return area

# 2.2 Aquí podrían ir las definiciones de clases

# 3. Bloque principal

print("Cálculo del área de un triángulo con la fórmula de Herón")

# Pedir al usuario los tres lados del triángulo
a = 3.0 # (Reemplazado para ejecución automática desatendida)
b = 4.0
c = 5.0

# Calcular el área con los valores introducidos
area = calcular_area_heron(a, b, c)

# Mostrar el resultado
print(f'El área triángulo de lados {a,b,c} es {area}')
```

2.2 Escritura de funciones.

2.3 Funciones en Python

Una **función** es un bloque de código que realiza una tarea específica y que podemos reutilizar varias veces.

Las funciones permiten **organizar mejor** el código y **evitar repeticiones**.

Se definen con la palabra clave **def**, seguida del nombre de la función, entre paréntesis sus argumentos, si los tiene, poniendo siempre al final los que tienen valores por defecto y terminando por ":"

```
def saludar():
    print("¡Hola!")
```

2.4 Documentación de funciones en Python

Cuando escribimos funciones, es importante **explicar qué hacen** para que otras personas (o incluso nosotros mismos en el futuro) puedan entenderlas sin dificultad.

Para ello se usan los **docstrings** (strings de documentación).

2.4.1 ¿Qué es un docstring?

Un *docstring* es un texto escrito entre **triple comillas** `""" """` que se coloca justo debajo de la definición de la función.

Sirve para describir:

1. Qué hace la función.
2. Qué parámetros recibe.
3. Qué devuelve (si corresponde).

2.4.2 Invocacion

Obtenemos lo que hay en el *docstring* escribiendo **help(nombre_funcion)**

```
[ ]: ### Ejemplo simple
def saludar():
    """
    Imprime un saludo por pantalla.
    """
    print("¡Hola!")

help(saludar)

saludar()
```

2.4.3 1. Funciones sin argumentos

Una función puede no necesitar información de entrada.
Por ejemplo, una que imprima una línea de separación:

```
[ ]: def linea():
    print("-----")

# Uso
linea()
linea()
```

2.4.4 2. Funciones con argumentos pero sin retorno

Podemos darle parámetros para hacerla más flexible. Por ejemplo, imprimir varias líneas:

```
[ ]: def imprimir_lineas(n):
    """
    Imprime una línea de separación ("-----") tantas veces como se indique.
    Uso:
```

```

    imprimir_lineas(n)

Parámetros:
    n (int): número de líneas a imprimir.
    """
    # El guion bajo (_) indica que la variable del bucle no se usa,
    # solo nos interesa repetir el bucle n veces.
    # Presta atención a la indentación (sangría de 4 espacios en Python).
    for _ in range(n):
        print("-----")

# Ejemplo de uso
imprimir_lineas(3)

```

2.4.5 3. Funciones con argumentos y valores por defecto

A veces queremos que, si el usuario no pasa un valor, la función use uno por defecto. Por ejemplo, imprimir siempre 3 líneas si no se indica otra cantidad:

```

[ ]: def imprimir_lineas(n=3):
    # el _ indica que en realidad la variable
    # sobre la que vamos a iterar no nos interesa, solo el numero de iteraciones
    # presta atencion a la indentacion
    for _ in range(n):
        print("-----")

# Uso
imprimir_lineas() #POR DEFECTO SI NO LE DECIMOS NADA, SIEMPRE 3 LINEAS
print('AHORA IMPRIMIMOS 6 LINEAS')
imprimir_lineas(6)

```

2.4.6 4. Funciones con argumentos y valor de retorno

Una función puede devolver un resultado en lugar de solo imprimir. Lo que devuelve se especifica con la palabra clave **return**. Por ejemplo, calcular el área de un triángulo rectángulo:

```

[ ]: def area_rectangulo(base, altura):
    area = base * altura / 2
    return area

# Uso
resultado = area_rectangulo(3, 4)
print("Área:", resultado)

```

2.4.7 5. Funciones con varios valores de retorno

Muchas veces, nos va a interesar que una función devuelva varias cosas ya que nos va a permitir ahorrar calculos. Esto se hace añadiendo en la línea de **return** las variables a devolver

```
[ ]: def propiedades_rectangulo(base, altura):

    area = base * altura
    perimetro = 2 * (base + altura)
    return area, perimetro

# Uso
a, p = propiedades_rectangulo(3, 4)
print("Área:", a)
print("Perímetro:", p)

#Si no asignamos a varios valores, devolvera una tuple.
Result=propiedades_rectangulo(3, 4)

print(type(Result))
print(Result[0])
print(Result[1])
```

2.5 Funciones lambda en Python

Las **funciones lambda** son **funciones pequeñas y anónimas** que se definen en una sola línea. Se usan cuando necesitamos una función **rápida y simple**, sin necesidad de definirla con **def**.

2.5.1 Sintaxis básica

lambda argumentos: expresión

Las funciones **lambda** son útiles para operaciones rápidas o cuando se usan como argumentos de otras funciones, como **map()**, **filter()** o **sorted()**.

No reemplazan funciones normales (def) si necesitamos código más complejo o docstrings.

```
[ ]: # Función lambda que suma 2 al número recibido
sumar_dos = lambda x: x + 2

# Uso
print(sumar_dos(5)) # salida: 7
```

2.6 Funciones lambda: aplicaciones con map() y filter()

Aunque las **funciones lambda** son pequeñas, son muy útiles cuando las usamos como **argumentos de otras funciones**.

Ejemplo 1: Usando map()

map() aplica una función a cada elemento de una lista.

```
[ ]: numeros = [1, 2, 3, 4, 5]

# Queremos sumar 10 a cada número
resultado = list(map(lambda x: x + 10, numeros)) #Sin necesidad de hacer un
↳ bucle y recorrer cada elemento de la lista.

print(resultado) # salida: [11, 12, 13, 14, 15]
```

Ejemplo 2: Usando filter()

filter() selecciona elementos de una lista que cumplan una condición.

```
[ ]: numeros = [1, 2, 3, 4, 5]

# Queremos quedarnos solo con los números pares.
# Usamos para ello el operador % (modulo)
pares = list(filter(lambda x: x % 2 == 0, numeros))

print(pares) # salida: [2, 4]

#Hazlo tu para los impares sabiendo quee si es impar entonces x % 2 es 1
```

3 Ejercicios prácticos: Scripts y funciones en Python

En estos ejercicios podrás poner en práctica todo lo aprendido sobre **scripts**, **funciones**, **doc-strings**, **lambda**.

3.1 Ejercicios sobre scripts

1. Crea un **script** que pida al usuario su nombre y edad, y luego imprima: Hola, **nombre**, el próximo año tendrás **edad+1** años.
 - Recuerda pedir los datos con `input()`.
 - Convierte la edad a número con `int()`.

```
[ ]: #tu codigo aqui
```

2. Escribe una función `perimetro_rectangulo(base, altura)` que **devuelva** el semiperímetro del rectángulo.
 - Llama a la función y guarda el resultado en una variable, luego imprímelo.

```
[ ]: #tu codigo aqui
```

3. Escribe una función `area_rectangulo(base, altura)` que **devuelva el área** del rectángulo.
 - Llama a la función y guarda el resultado en una variable, luego imprímelo.

```
[ ]: #tu código aquí
```

4. Escribe una función 'propiedades_rectangulo(base, altura)' que **devuelva el área y el perímetro reusando las dos funciones anteriores**. Añade docstrings** a esta función explicando:

- Qué hace la función.
- Qué parámetros recibe.
- Qué devuelve (si aplica).
- Llama al help de la función para que muestre el **docstring**
- Llama a la función y guarda los resultados en dos variables (a, p), luego imprímelos.

```
[ ]: #tu código aquí
```

3.2 Reutilización de Funciones en Módulos Propios (mylib.py)

En proyectos de investigación del CSIC, es habitual organizar funciones comunes en archivos de Python (.py) dentro de nuestro directorio de trabajo para importarlas en múltiples cuadernos o scripts.

```
from mylib import print_line
```

```
print_line() # Imprime una línea de separación desde nuestro propio módulo
```

```
[ ]: # Crear un archivo de módulo local mylib.py dinámicamente si no existe
mylib_content = "# mylib.py - Módulo auxiliar para el curso CSIC\ndef
↳ print_line(caracter='-', longitud=40):\n    '''Imprime una línea de
↳ separación personalizada.'''\n    print(caracter * longitud)\n"

with open("mylib.py", "w", encoding="utf-8") as f:
    f.write(mylib_content)

# Importar y ejecutar la función desde nuestro módulo mylib
from mylib import print_line

print("Probando la función print_line() desde el módulo propio 'mylib':")
print_line("=", 45)
print("  Investigación Abierta - CSIC España")
print_line("=", 45)
```

3.3 Solucionario Integrado de los Ejercicios del Módulo 03

A continuación se muestra la solución de referencia para los ejercicios de diseño de funciones y docstrings.

```
[ ]: # SOLUCIÓN DE REFERENCIA - MÓDULO 03

def perimetro_rectangulo(base, altura):
    """Devuelve el semiperímetro de un rectángulo (base + altura)."""
    return base + altura

def area_rectangulo_base(base, altura):
    """Devuelve el área de un rectángulo (base * altura)."""
    return base * altura

def propiedades_rectangulo_completo(base, altura):
    """
    Calcula el área y el perímetro de un rectángulo reutilizando funciones
    ↪ previas.

    Parámetros:
        base (float): Longitud de la base del rectángulo.
        altura (float): Longitud de la altura del rectángulo.

    Devuelve:
        tuple: (área, perímetro)
    """
    area = area_rectangulo_base(base, altura)
    perimetro = 2 * perimetro_rectangulo(base, altura)
    return area, perimetro

print("=== VERIFICACIÓN DEL SOLUCIONARIO DE FUNCIONES ===")
print("1. Inspección de docstring con help():")
help(propiedades_rectangulo_completo)

a_val, p_val = propiedades_rectangulo_completo(5.0, 3.0)
print("")
print(f"2. Resultados para base=5.0 y altura=3.0 -> Área: {a_val}, Perímetro:
    ↪ {p_val}")
```